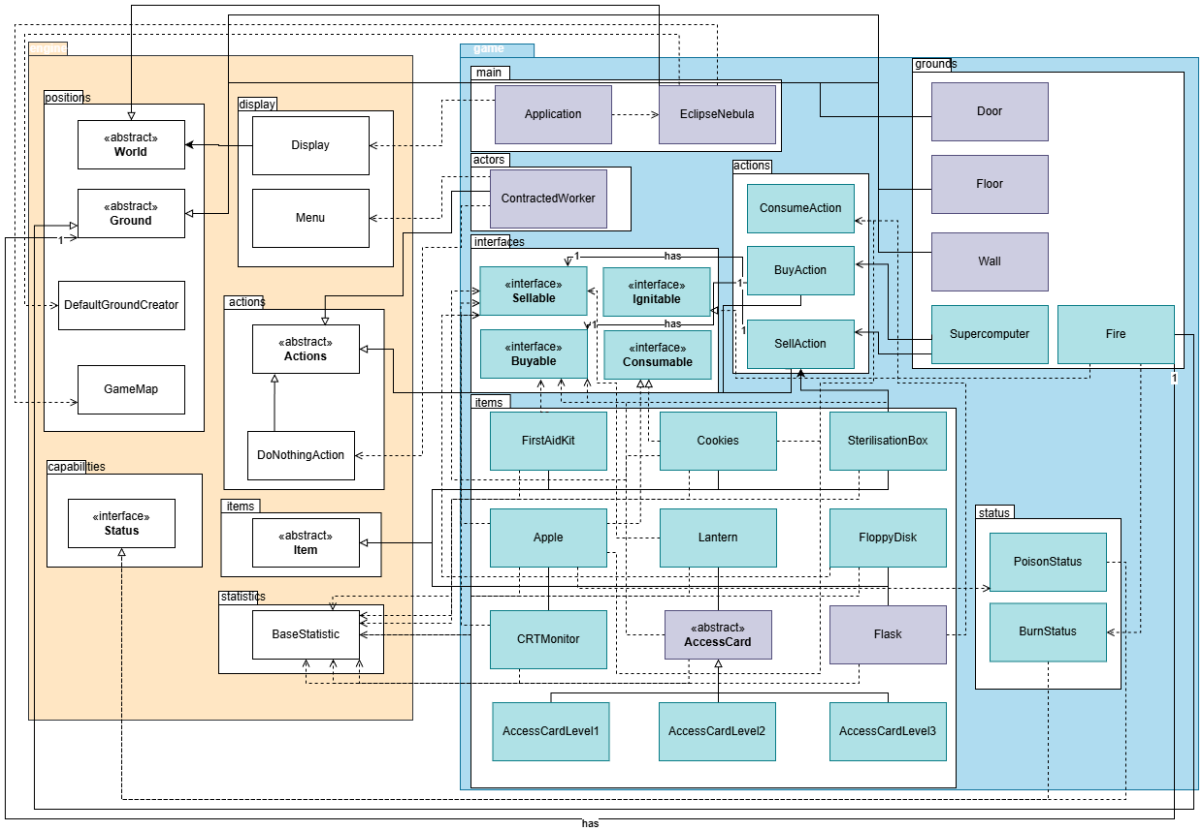
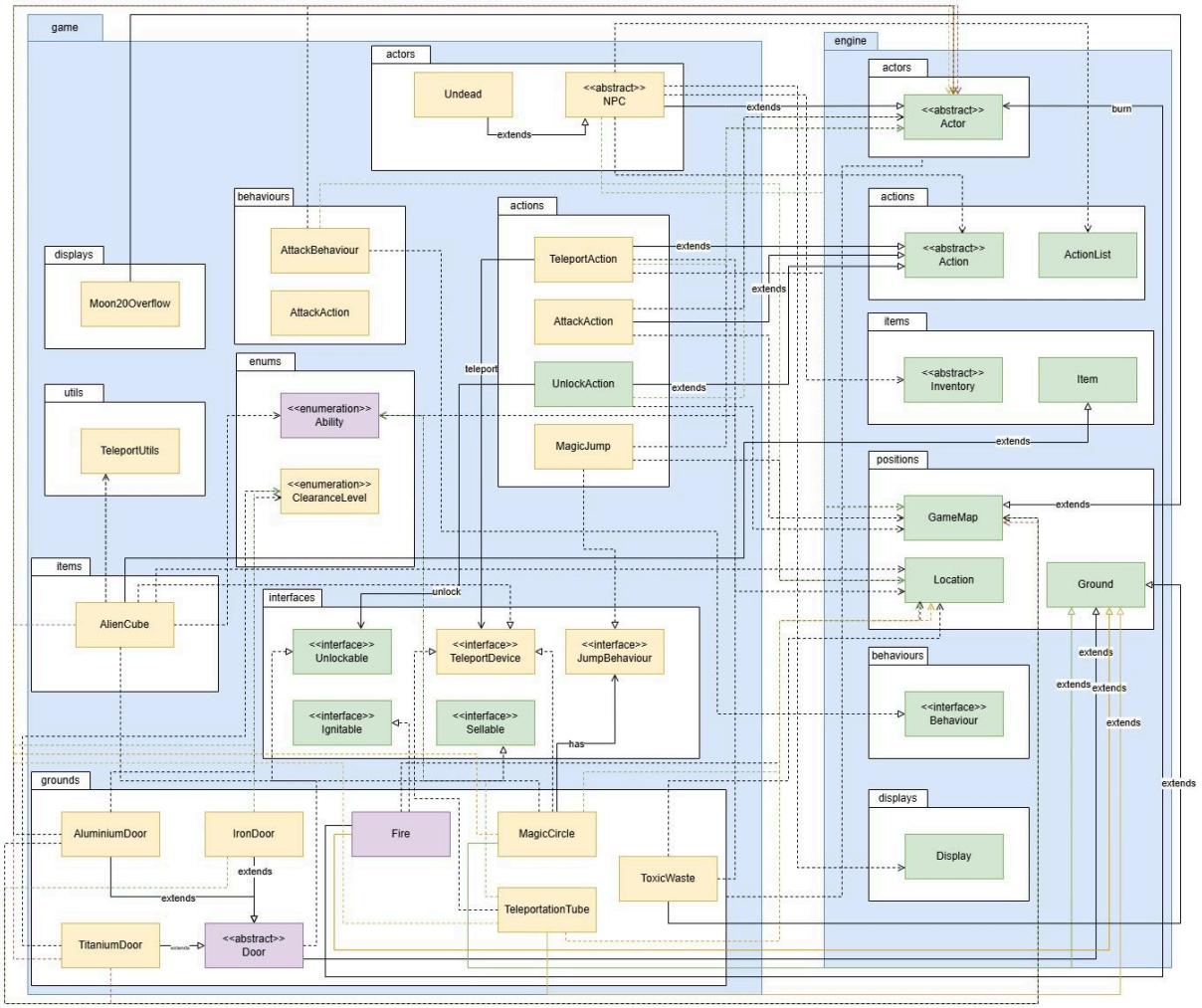
FIT2099 Assignment 2 Design Documentation

Design Diagrams

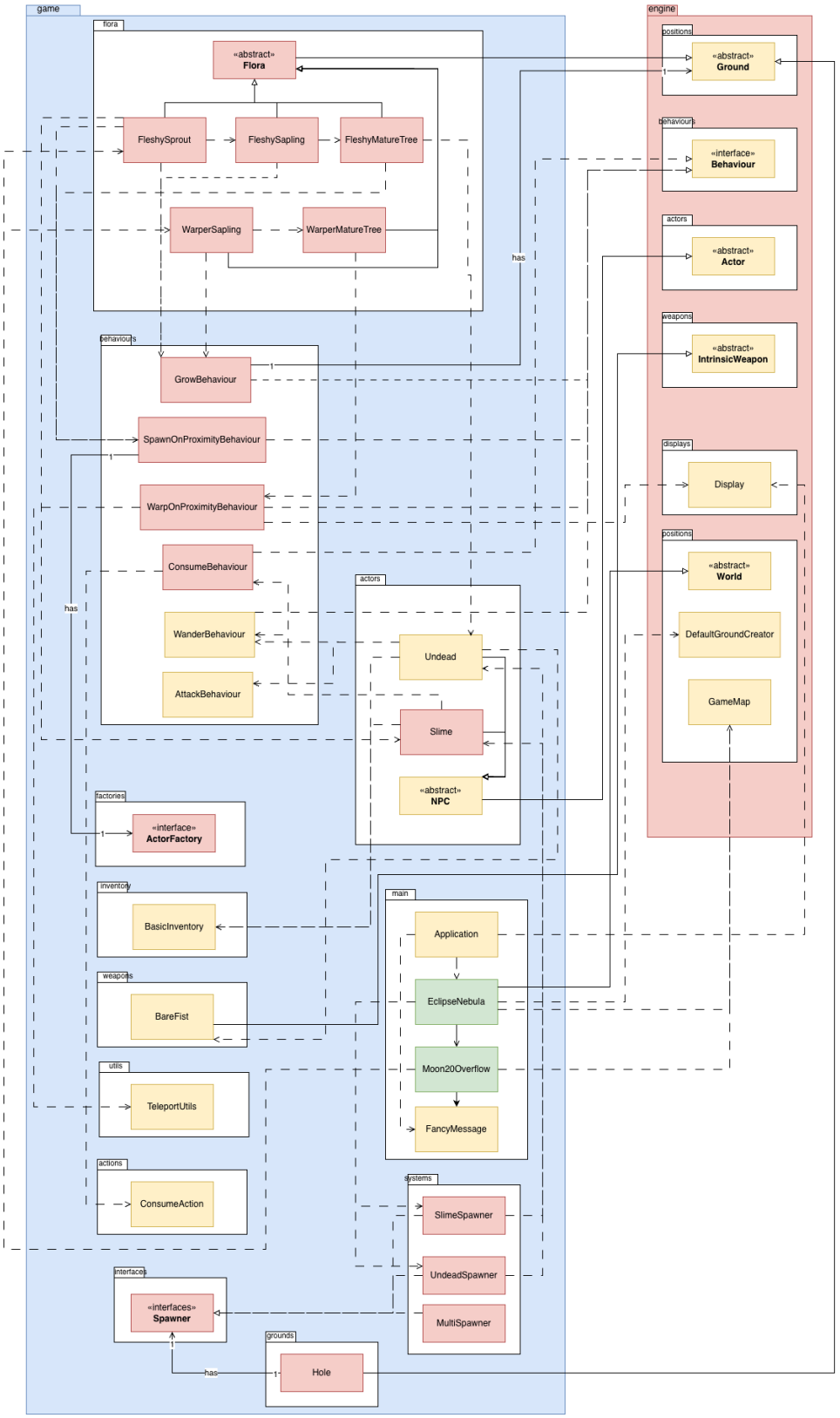
REQ 1 UML Class Diagram:



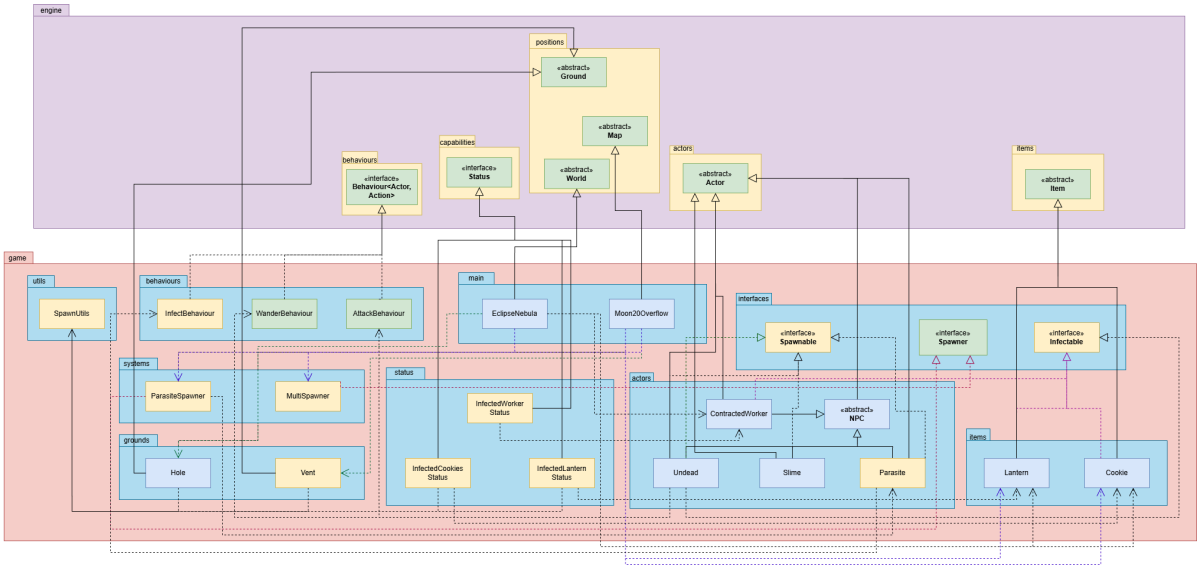
REQ 2 UML Class Diagram:



REQ 3 UML Class Diagram:



REQ 4 UML Class Diagram:



Design Documentations

REQ 1

A: Object Hierarchy & Framework Alignment

Design 1: Isolated Standalone Entities. Creating concrete classes for game items without inheriting from the engine's Item class

Pros	Cons
the user can define exactly how an object looks and works without following the engine's rules.	The items won't work with the rest of the game. The Map and the Inventory only recognize things that are officially an Item. This would break the Liskov Substitution Principle (LSP).
This design offers better memory efficiency. The user avoids inheriting unused variables or methods from the parent class.	

Design 2: Integrated Engine Subclassing. Extending the engine's abstract Item class for all custom game items

Pros	Cons
This ensures that the First Aid Kit and Apple are recognized by the game. It also follows the don't repeat rule because we don't have to rewrite code for things like picking up or carrying items.	The code is tied to the engine. If the game engine changes, we might have to update all our items.
The custom items can be treated as generic Items by the engine but as Sellables by the Supercomputer. Results in polymorphism.	

B: Interface-Driven Transaction Architecture

Design 1: Putting all logic inside the Action classes

Pros	Cons
Easy to find all prices in one or two files. the user can look at SellAction and see every price and every side effect for the entire game in one file.	Very messy. Every time the user sells an item, the code has to check if it's an Apple, a Lantern, or a Monitor. This makes the code weak. If a user adds a new item, the user has to change the SellAction code, which often leads to mistakes.
Offers simplicity. For very simple items (like a Floppy Disk that just gives 1 credit), this is faster to write.	This violates the Open-Closed Principle, as the class is never "closed" for modification. Hard to test code. To test if an Apple poisons a worker, the user has to run the entire SellAction instead of just testing the Apple's logic. Also has a high complexity.

Design 2: Decentralized Polymorphic Contracts

Pros	Cons
The BuyAction and SellAction are very simple. They just command the item to run their own trade logics. This follows the Dependency Inversion Principle. The Monitor handles its own healing, and the Access Card handles its own blood sample logic.	It spreads the logic out. To see what a Lantern does when sold, the user have to open the Lantern class instead of the SellAction class.art.
It offers good scalability also. The user can add 1,000 new items with 1,000 different effects, and the user will never have to change the BuyAction or SellAction code again.	End up with having more files in the project.

C: Capability & State Management

Design 1: Primitive Flagging (String/Boolean). Using simple True/False flags or Text (Strings) inside classes

Pros	Cons
Quick to implement for a very small number of features. Good for small features that won't be reused. No need to create extra classes or Enum files.	Easy to make typos. If the user has many items that cause poisoning, the user would have to copy and paste the same code everywhere. This makes the system hard to manage.
	Might cause logic duplication. For example, if both the Lantern and Fire cause "Burning," the user'll end up writing the same damage logic in two different places.

Design 2: Modular Capability & Status System. Using Centralized Enumerations (Ability) and Status classes

Pros	Cons
The Apple knows it is poisonous, but the PoisonStatus class knows how to deal the damage. This follows the Single Responsibility Principle (SRP).	Requires setting up a dedicated package structure for statuses and abilities.
Offers reusability, With BurnStatus, any item in the game can use it. The compiler also catches typos in Enums, resulting in type safety being ensured.	

REQ 1 Final Justification:

Design 2 is chosen for all components because it builds a "Future-Proof" game.

Scalability via SOLID: By using Sellable and Buyable interfaces, the game is Open-Closed. We can add a CRT Monitor that heals on sale or a Floppy Disk that glitches credits without ever changing the SellAction code. The "Action" is the manager, but the "Item" is the expert on its own behavior. Compatibility via LSP: Extending the Item class is the only way to ensure our items work with the engine's map and inventory systems. It allows us to focus on the unique "Eclipse Nebula" features rather than rewriting basic inventory logic. Clean Logic via Separation of Concerns: Using Ability and Status classes ensures that every class has a Single Responsibility. The Lantern is responsible for lighting and selling, while the BurnStatus is responsible for handling damage over time. This separation makes the code easier to debug and test.

Type Safety and Capability-Based Logic is also achieved. Using Abilities and Statuses instead of instanceof or "magic strings" prevents silent failures. When a Lantern leaks oil, it checks currentLocation.getGroundAs(Ignitable.class). It doesn't care if the ground is a Floor or a Supercomputer; it only cares if the ground can be set on fire. The use of actor.addStatus(new BurnStatus(...)) ensures that damage logic isn't repeated. The BurnStatus class is the only place in the code that needs to know how to subtract health every turn. In short, Design 2 creates a professional foundation. It treats the game like a collection of independent, smart objects that know how to interact with each other, rather than a giant, messy pile of hard-coded instructions.

Ultimately, Design 2 turns the code from a "script" into a system of independent agents. The Supercomputer terminal (≡) stays simple and reliable, while the dangerous, unpredictable nature of the Eclipse Nebula scrap is contained safely within each item's class. This creates a professional, modular foundation that is easy to debug, easy to test, and ready for any future requirements the Company might throw at the workers.

REQ 2

A: Inheritance and Separate Implementation

Design 1: Three Separate Concrete Classes

Pros	Cons
Changes to the Iron Door logic will not accidentally break the other doors.	We will likely copy and paste the “isUnlocked” variable and the “canActorEEnter” logic to each door class.
	Every time we add a foot, we might need to update the “UnlockAction” with more enumeration checks.

Design 2: Inheritance from Abstract class

Pros	Cons
Logic for checking clearance levels and managing the “unlocked” state is written once in the parent class.	Any change to the base “Door” class automatically affects all subclasses.
The “UnlockAction” can interact with any “Door” object without needing to know if it is Aluminium, Iron or Titanium.	
Adding a fourth door type requires only defining the new “unlock()” effect, not rewriting the whole class.	

B: Interface Implementation for Teleportation

Design 1: TeleportationTube as a Standalone Class

Pros	Cons
No need to define an interface or implement mandatory methods. This follows the KISS (Keep It Simple, Stupid) principle for small-scale projects with only one device.	Every time a new teleport device is added, the “TeleportAction” must be modified,
The developers reading the code, the connection between the Action and the Tube is explicit and lacks the "indirection" of an interface.	The “TeleportAction” becomes “glued” to the “TeleportationTube”. If we change a method name in the Tube, the Action class breaks immediately.
	We may end up writing multiple different Action classes that do mostly the same thing.

Design 2: TeleportationTube implements TeleportDevice

Pros	Cons
The “TeleportAction” can treat a “TeleportationTube”(Ground), an “AlienCube”(Item), and a “MagicCircle” (Ground) identically. Adding new devices requires zero changes to the Action class.	Requires managing an additional interface file and ensuring all implementing classes follow the exact method signatures, which can be overkill for very simple systems.
The specific tube logic stays inside the Tube class. The Action only knows “how to start”, instead of “what happens”	All devices are forced to use the same parameters, even if a specific device (Magic Circle) doesn’t need all of them.
High-level game logic depends on the “TeleportDevice” abstract, not the	

concrete “teleportationTube” class.	

C: Separation of MagicJump and TeleportAction

Design 1: Single Monolithic Action

Pros	Cons
Fewer classes to manage initially.	The Action becomes responsible for movement, fire, flasks, malfunctions, and toxic waste logic, which violates the SRP.
No need for interfaces like “JumpBehaviour’	We may end up copying and pasting the “move actor” code into multiple if-else branches.

Design 2: Separated Interfaces

Pros	Cons
“MagicJump” only cares about magic logic, while “TeleportationTube” only cares about the tube logic.	Slightly more files and initial setup in the interfaces package.
If a Flask doesn’t spawn, we may know exactly which file to look in, which makes debugging easier.	Requires careful coordination between the Action and the Behaviour implementations.

Final Design Justification REQ2:

The final design of REQ2 utilizes Design 2 for all core components, prioritizing a flexible and decoupled architecture to handle the moon's hazardous teleportation and security systems.

By implementing an Inheritance–based hierarchy for the tiered security system, the design ensures that "AluminiumDoor", "IronDoor" and "TitaniumDoor" all leverage a shared "Door" superclass. This effectively applies the Liskov Substitution Principle (LSP), as the engine's movement and action handlers can interact with any door type generically. This centralizes clearance-level validation and "unlocked" state management, significantly reducing code duplication and ensuring that higher-tier access cards naturally satisfy lower-tier requirements.

To manage the distinct teleportation methods, I utilized an Interface-Driven Transaction Architecture by having the "TeleportationTube", "AlienCube" and "MagicCircle" implement the "TeleportDevice" interface. This architectural choice enforces the Dependency Inversion Principle (DIP), as the "TeleportAction" depends on an abstraction rather than concrete moon-specific classes. This allows the system to remain Open-Closed, where adding a new teleportation method will only require a new implementation of the interface rather than a modification of the central action logic.

Furthermore, the implementation of specialized behavioural strategies, such as MagicJump, demonstrates a strong commitment to the Single Responsibility Principle (SRP). By separating the "arrival behaviour" from the physical device, we ensure that the environmental consequences, such as spawning a "Flask" or igniting "Fire", are encapsulated within specific strategy classes. This separation of concerns creates a robust and maintainable foundation that allows the game to scale with complex environmental interactions without introducing technical debt.

REQ 3

A: Base Flora structure and engine integration

Design 1: Make the abstract class Flora extend the abstract Actor or NPC class so it can access the engine's behaviour queue to execute actions like growing

Pros	Cons
Reuse the existing NPC class, reducing the need to implement separate logic for triggering actions each round.	Violates Liskov Substitution Principle, since a tree is not truly an Actor, Trees do not have inventories or make intentional decisions
	Actors cannot occupy the same tile, so additional collision handling logic would be required to let workers move around trees.

Design 2: Create an abstract Flora class that extends Ground, and give it a TreeMap of Behaviour (Ground, Boolean) objects executed during the map's tick() phase.

Pros	Cons
The tree has behaviours rather than inheriting a list of irrelevant Actor that it will never use.	Requires writing custom execution and iteration logic inside the Flora tick method
Accurately models the trees as Ground type rather than a conscious Actor	

B: Dynamic Creature Spawning Mechanics

Design 1: Hardcode the instantiation of specific enemies directly inside the behaviour (e.g., calling new Slime() inside a SpawnSlimeBehaviour).

Pros	Cons
Simple and direct implementation.	Causes "Class Explosion." You would need a separate behaviour class for every single enemy in the game.
	Tightly couples the environmental behaviour to concrete enemy classes.

Design 2: Implement the Factory Pattern via an ActorFactory interface. Create a generic SpawnOnProximityBehaviour that accepts a factory in its constructor and calls factory.createActor().

Pros	Cons
Very flexible design. SpawnOnProximityBehaviour determines when and where enemies spawn, while the Factory decides what enemy to create.	Adds extra abstraction, so developers need to understand how inner classes are passed into constructors.
Prevents crashes by creating a new enemy instance every time spawning happens.	

C. Flora Growth Mechanism

Design 1: Handle age tracking and replacement logic directly inside the concrete tree's tick() method (e.g., if (age > 20) location.setGround(...)).

Pros	Cons
	Causes duplicated logic because each plant stage must handle its own age tracking and probability calculations.

Design 2: Implement a generic GrowBehaviour that accepts the required turns, percentage chance, and the Ground object of the next stage.

Pros	Cons
Code reuse (DRY). A single class manages all probability and temporal math for every growing entity in the game.	

D. Player detection

Design 1: Loop through exits and use if (actor instanceof ContractedWorker) to determine if the target should trigger a spawn or warp.

Pros	Cons
Very quick and simple to implement.	Violates Open-Closed Principle. If a new player class is added, the flora logic must be rewritten to target them too.
	Creates tight coupling. It forces the low-level environmental classes (Flora) to depend directly on high-level concrete actor classes (ContractedWorker). Relying heavily on instanceof.

Design 2: Loop through exits and use if (actor.hasAbility(ActorTags.WORKER)) to dynamically query the target's alignment.

Pros	Cons
Highly scalable. Any future entity can simply add ActorTags.WORKER to its capability list during instantiation.	Requires careful initialization. If a new worker class is added but the capability tag is not set in the constructor, trees may fail to detect it silently.
Uses Dependency Inversion by decoupling hazard logic from specific actor classes. The tree behaviour relies only on an abstract capability (enum).	

E. Structural separation automated logic

Design 1: Hardcode the logic for movement, attacking, growing, and spawning directly inside the playTurn() method of the specific Actor or the tick() method of the specific Ground.

Pros	Cons
Easy for beginners because all logic for an entity stays in one file.	Leads to large classes (e.g. playTurn() becoming very long), violating the Single Responsibility Principle (SRP).
	Reduces code reuse, since shared behaviours like wandering must be copied into multiple classes, violating DRY.

Design 2: Utilize the Behaviour interface to extract specific automated actions into independent classes (e.g., WanderBehaviour, SpawnOnProximityBehaviour).

Pros	Cons
Separating what an entity is from how it behaves, which enforces the Single Responsibility Principle.	Increases class count, making the project more fragmented and requiring developers to trace logic across multiple files.
High reusability, since a single WanderBehaviour can be used across different entities like Slime, Spider, or NPCs, supporting the Open-Closed Principle (OCP).	

F. Management of Execution Order and Single-Threaded Logic

Design 1: Manage multiple behaviours using a sequence of if-else statements directly inside the entity's turn logic

Pros	Cons
The execution flow is easy to follow in a top-to-bottom manner for that specific entity.	Violates the Open-Closed Principle because adding new behaviours requires modifying existing if-else logic in the entity.
	Increases risk of bugs since changes to the core logic can unintentionally affect existing behaviour.

Design 2: Manage behaviours using a sorted TreeMap<Integer, Behaviour>. The entity iterates through the map in ascending priority order. The first behaviour to successfully return a valid action (or true) halts the loop.

Pros	Cons
Highly extensible, as new behaviours can be added at runtime without modifying the execution loop.	Requires careful management of integer priority keys to maintain correct behaviour order
Strictly enforces the single-action-per-turn rule, since the loop exits immediately after a successful behaviour executes.	

G. Implementation of concrete plant stages

Design 1: Concrete plants override Ground and implement tick() directly

Pros	Cons
Removes the need for an abstract class, reducing the number of files. Allows full structural freedom for each plant class.	Severe code duplication, violating DRY, as each plant reimplements similar if-else logic for single-threaded execution.
	Harder to maintain global behaviour rules, since changes require updates across all concrete plant classes.

Design 2: Plants extend Flora and inject behaviours using a constructor

Pros	Cons
Highly DRY, with shared tick() logic centralized in the Flora superclass.	
Promotes clean Liskov Substitution, allowing all plant types to be treated uniformly as Flora by the engine without unexpected behaviour differences.	

FINAL DESIGN JUSTIFICATION REQ3:

A key design decision is defining Flora as a subclass of Ground rather than Actor. This avoids conflicts with the engine's Actor-based collision system, allowing smooth navigation while maintaining the Liskov Substitution Principle, since flora does not inherit unnecessary actor-related features like inventories or action logic.

Classes such as FleshySprout and WarperTree extend the abstract Flora class, promoting strong code reuse through the DRY principle. Instead of rewriting complex turn-based logic, these classes simply add their required behaviours, such as GrowBehaviour and SpawnOnProximityBehaviour, into the parent class's TreeMap during construction. As a result, the concrete classes follow the Single Responsibility Principle, since they only define which behaviours they have, while the Flora superclass handles behaviour execution.

Behavioural logic is separated into modular Behaviour classes such as GrowBehaviour, AttackBehaviour, and SpawnOnProximityBehaviour. These are managed through prioritized TreeMap structures, which act as a central execution system. This setup enforces a single-action-per-turn rule for plants and keeps the system open for extension but closed for modification under the Open-Closed Principle.

The system also applies Dependency Inversion through the ActorFactory interface, which decouples the environment from concrete enemy classes such as Slime or Undead. This avoids tight coupling and makes spawn logic easily extensible. Capability tags via hasAbility() further replace unsafe instanceof checks with more flexible abstractions.

REQ 4

A. Polymorphic Infection Effect

Design 1: Hard-coding infection branches inside the Parasite class

Pros	Cons
Centralizes the reaction logic of every host, making it easier to read at a glance.	Violates the Open-Closed Principle (OCP) and project rules by requiring instanceof or class checks to identify hosts.
Avoids the creation of new interfaces within the game.interfaces package.	Tightly couples the parasite class to every host type, requiring code edits for any new infectable entities.

Design 2: Polymorphic delegation via the Infectable interface

Pros	Cons
Uses the Infectable interface to delegate host-specific effects, ensuring the Parasite never needs to know concrete types.	Results in a minor increase in the total number of interfaces in the game.interfaces package.
Treats Actors and Items uniformly by leveraging asCapability on any GameEntity.	Distributes the reaction logic across various host classes rather than maintaining it in a central location.
Allows for the addition of new hosts without modifying the Parasite or InfectBehaviour.	

B: Spawn-time Environmental Reaction

Design 1: Duplicating reaction logic at every spawn site

Pros	Cons
Avoids introducing new abstractions; reactions are performed inline at every spawn site.	Violates the DRY principle as logic must be re-implemented at every site, increasing the risk of missing reactions in future updates. .
Localizes control flow by placing reaction effects near the spawn event.	Makes maintenance difficult; global specification changes must be manually applied to every individual spawn site.

Design 2: Spawnable capability dispatched through SpawnUtils

Pros	Cons
Each creature manages its own unique spawn reaction within an onSpawn(Location) method.	Requires a comprehensive refactoring of all existing and new spawn sites to call SpawnUtils.spawn.
Creating a canonical entry point makes it structurally impossible for a spawn site to forget a reaction.	Increases the project's surface area by adding a utility class and a new interface.
Guarantees the reaction fires regardless of the specific trigger, aligning with the project requirements.	

C: Hole and Vent Creature Roster

Design 1: One subclass of Hole / Vent per map

Pros	Cons
Allows map-specific hole classes to independently override tick logic or spawn intervals. .	Violates the DRY principle as logic must be re-implemented at every site, increasing the risk of missing reactions in future updates. .
	Adding a third map later would require yet another near-identical subclass, scaling badly with content.

Design 2: Single Hole / Vent class parameterized by a Spawner

Pros	Cons
Uses a single class for all maps, handling differences through constructor-injected Spawner arguments.	Requires the implementation of a small Spawner class hierarchy.
Supports composite spawning via MultiSpawner, allowing holes to spawn multiple creature types blindly.	Shifts information out of the class name, requiring developers to trace injected objects to see what a hole spawns
Simplifies world setup, as adding a hole for a new map becomes a one-line change.	

D: NPC Death Cleanup From Passive Damage

Design 1: Patching every damage source to call unconscious

Pros	Cons
Links damage and cleanup together, ensuring the source that kills also handles removal.	Scatters cleanup logic across many unrelated classes, making it easy to miss a source and re-introduce bugs.
Enables contextual death messages based on the damage source.	Extends the scope of changes into pre-existing game code for items like Fire and ToxicWaste.
	Place the burden on future contributors to remember to call unconscious(map).

Design 2: Centralized cleanup at the start of NPC.playTurn

Pros	Cons
Provides a single place fix in the base class that all current and future NPCs inherit automatically.	The corpse remains on the map for up to one tick after death before being removed on its next turn.
Keeps damage sources simple and focused solely on dealing damage.	Requires a DeathReportAction subclass to handle the death message within the engine's pipeline.
Maintains consistency by mirroring the cleanup pattern used by ContractedWorker.	

Final Design Justification REQ4

The project uses Design 2 for all parts of REQ 4 to prioritize long-term maintenance and solid design principles. The Infectable interface is the most important part of the infection system. By checking for this interface, the Parasite follows project rules by avoiding instanceof checks and can infect different entities like workers or items in the same way. This follows the Dependency Inversion Principle (DIP) because the logic relies on an abstraction, and the Single Responsibility Principle (SRP) because each host manages its own reaction.

The Spawnable capability and SpawnUtils ensure that spawn reactions happen every time, as the project spec requires. By putting reactions inside onSpawn() methods, new creatures can be added later without changing existing code, following the Open-Closed Principle (OCP). Using one SpawnUtils helper guarantees no reaction is ever skipped.

For map structures, using a Spawner is better than creating many subclasses because the differences between holes are just data. This approach uses the Strategy pattern to define what spawns at world-setup time without making the class hierarchy too big. Passing the spawn interval through the constructor also allows different maps to have different speeds without needing new classes.

Finally, putting death cleanup in NPC.playTurn fixes the ghost turn bug for all NPC dying from passive damage like fire or poison. This single fix applies to all current and future NPCs, which is a perfect example of the DRY principle. To keep things simple, the Parasite uses a basic WanderBehaviour if no target is nearby, meeting all requirements without extra complexity.